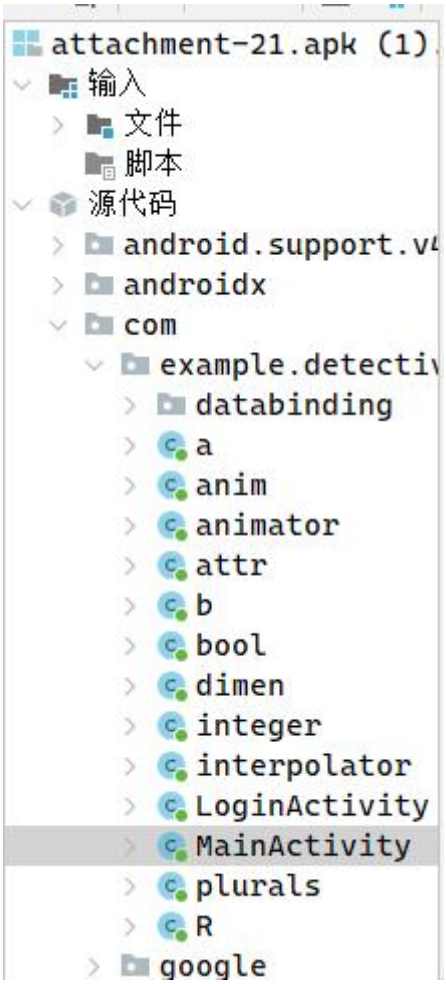
用 jadx 打开



找到 main 函数

```
CompatActivity;
binding.ActivityMainBinding;

s AppCompatActivity {
binding;
;

omJNI(String str);

tive");

t.app.FragmentActivity, androidx.activity.ComponentActivity, androidx.core.app.ComponentActivity, android.app.Activity
le bundle) {

te = ActivityMainBinding.inflate(getLayoutInflater());

tRoot());
.binding.editTextFlag;
ding.button;
n;
(new View.OnClickListener() { // from class: com.example.detective.MainActivity.1
view.View.OnClickListener
View view) {
this.JFormat(MainActivity.this.flagEditText.getText().toString())) {
t(MainActivity.this, "Congratulations, you are right!", 1).show();

t(MainActivity.this, "PITY", 0).show();



s changed from: private */
g str) {
&& (str.length() + 1) % 2 != 0 && str.substring(0, 5).equals("ISCC{") && str.charAt(str.length() - 1) == '}' && stringFromJNI(a.a(str.substring(5, str.length() - 1))).equals("B05C57115B5F500823495941 2F");
```

找到比较字符串

写了个脚本直接出 flag

```
E:\Users\Jiker\PycharmProjects\jiker\.venv\Scripts\python.exe E:\Users\Jiker\PycharmProjects\jiker\luogu\27.py
[16, 24, 68, 73, 63, 9, 86, 89, 16, 94, 87, 1, 64, 78]
Cipher : Cp!;Sf52C62s,!
Plain  : B@skRvl%
ISCC{B@skRvl%}
```

```python
def _hex_from_cipher(cipher: str) -> str:
    """A.c 的逆操作：字符 ➜ 4 位十六进制，去掉前导 '00'"""
    hex_str = ''.join(f'{ord(ch):04x}' for ch in cipher)
    return hex_str[2:] if hex_str.startswith('00') else hex_str

def _undo_padding(hex_str: str) -> str:
    """
    逆操作：去掉每 2 位后强插的 '00'（若剩余长度 %4 == 2 说明末尾少一个 '00'），
    得到插针/交换前的串 replaced
    """
    if len(hex_str) % 4 == 2:
        hex_str += '00'             # 还原被截掉的最后两个 0
    return ''.join(hex_str[i:i+2] for i in range(0, len(hex_str), 4))

def _undo_pair_shuffle(replaced: str) -> str:
    """
    逆操作：解析 2-字节或 4-字节(…'21') 片段，复原 sb2+sb 的顺序
    """
    out, i = [], 0
    while i < len(replaced):
        # 4 字节模式：c2 c1 2 1
        if i + 3 < len(replaced) and replaced[i+2:i+4] == '21':
            c2, c1 = replaced[i], replaced[i+1]
            out.extend([c1, c2])
            i += 4
        else:                       # 2 字节保持原序
            out.extend(replaced[i:i+2])
            i += 2
    return ''.join(out)

def _split_and_fix_zero(concat: str) -> str:
```

```
    """
    按 sb2 | sb 切分，再把被替换成 '3' 的零位改回 '0'。
    sb 负责偶数索引 (0,2,4…) ，替换条件  idx==0 或 idx%3==0
    sb2 负责奇数索引 (1,3,5…) ，替换条件  idx==1 或 (idx-1)%3==0
    """
    half = len(concat) // 2
    sb2, sb = concat[:half], concat[half:]

    # sb 复原 0 → 3
    sb = ''.join('0' if ch == '3' and (idx == 0 or idx % 3 == 0) else ch
                 for idx, ch in enumerate(sb))
    # sb2 复原 0 → 3
    sb2 = ''.join('0' if ch == '3' and (idx == 1 or (idx - 1) % 3 == 0) else ch
                  for idx, ch in enumerate(sb2))

    # 交叉还原到原 16 进制序列
    res = []
    even = odd = 0
    for idx in range(len(concat)):
        if idx % 2 == 0:           # 偶数位来自 sb
            res.append(sb[even]);  even += 1
        else:                     # 奇数位来自 sb2
            res.append(sb2[odd]);  odd += 1
    return ''.join(res)

def _hex_to_plain(hex_str: str) -> str:
    """
    根据 A.b 的编码规则:
    - 如果以 '0' 开头则为 3 位组 (0xx) ➜ ASCII/单字节
    - 否则为 4 位组 ➜ Unicode (xxxx)
    """
    out, i = [], 0
    while i < len(hex_str):
        if hex_str[i] == '0' and i + 2 < len(hex_str):
            out.append(chr(int(hex_str[i+1:i+3], 16)))
            i += 3
        else:
            out.append(chr(int(hex_str[i:i+4], 16)))
```

```
            i += 4
    return ''.join(out)

def decrypt(cipher: str) -> str:
    """顶层接口：密文 ➜ 明文"""
    step1 = _hex_from_cipher(cipher)
    step2 = _undo_padding(step1)
    step3 = _undo_pair_shuffle(step2)
    step4 = _split_and_fix_zero(step3)
    return _hex_to_plain(step4)

# ------------------------ 示例 ------------------------
if __name__ == "__main__":

    xor_key = [0x53, 0x68, 0x65, 0x72, 0x6C, 0x6F, 0x63, 0x6B]
    res = "1008444F0F2B5253600444475E4E534A"      # 填

    res_chr=[]
    for i in range(0,len(res),2):
        res_chr.append((int(res[i:i+2],16)))
    print(res_chr)

    flag = []
    for i in range(len(res_chr)):
        flag.append((res_chr[i]^xor_key[i%(len(xor_key))]))

    sample =bytes(flag).decode()
    print("Cipher :", sample)
    print("Plain  :", decrypt(sample))
    print("ISCC{"+decrypt(sample)+"}")
```